

Mini-Test 2 (formatif) - 420-211

Département d'Informatique | Applications Web

21 avril 2026

1 Revue de code (40%)

Exo 1 : Déstructuration et Valeurs par défaut

```
1  const AfficherNote = ({ valeur, matiere = "Informatique" }) => {  
2    return <h1>{matiere} : {valeur} </h1>;  
3  };  
4  
5  // Utilisation dans l'App :  
6  <AfficherNote valeur={18} />
```

Qu'est-ce qui sera rendu à l'écran par ce composant ?

Exo 2 : Logique booléenne (Court-circuit)

```
1  const admin = false;  
2  const invite = "Visiteur";  
3  const access = admin || invite || "Anonyme";  
4  
5  console.log(access);
```

Quelle valeur sera affichée dans la console ?

Exo 3 : Hook useState et Ternaire

```
1  const [view, setView] = useState("grid");  
2  
3  const swap = () => setView(view === "grid" ? "list" : "grid");  
4  
5  // L'utilisateur clique une fois sur le bouton :  
6  <button onClick={swap}>Changer</button>
```

Si l'état initial est "grid", quelle sera la valeur de **view** après un clic ?

Exo 4 : Rendu conditionnel et opérateur &&

```
1  const Panier = ({ nbArticles }) => {  
2    return (  
3      <div>  
4        {nbArticles && <p>Vous avez des articles</p>}  
5      </div>  
6    );  
7  };  
8  
9  // Appel du composant avec zéro article :  
10 <Panier nbArticles={0} />
```

Que s'affichera-t-il dans le navigateur ?

2 Bonnes Pratiques et Concepts React (30%)

- Question 5 : Utilisation des listes et de la propriété **key**

Lors de l'utilisation de `.map()` pour générer une liste de composants, quel est le rôle principal de la propriété **key** ?

- (A) Définir le style CSS de chaque élément de la liste.
- (B) Servir d'index pour trier la liste par ordre alphabétique.
- (C) Aider React à identifier quels éléments ont changé, été ajoutés ou supprimés.
- (D) Permettre à l'utilisateur de cliquer sur les éléments.

- Question 6 : Dépendances du hook **useEffect**

Si vous utilisez une variable **data** à l'intérieur d'un **useEffect**, mais que vous laissez le tableau de dépendances vide `[]`, que se passera-t-il ?

- (A) Le code s'exécutera en boucle infinie.
- (B) Le **useEffect** utilisera toujours la valeur initiale de **data** (capture obsolète).
- (C) React affichera une erreur fatale et arrêtera l'exécution.
- (D) La variable **data** sera automatiquement ajoutée au tableau par le navigateur.

- Question 7 : Le bloc **finally**

Dans une structure **try...catch...finally**, quel est l'usage recommandé du bloc **finally** lors d'un appel API ?

- (A) Pour relancer l'appel si le premier a échoué.
- (B) Pour forcer l'utilisateur à se reconnecter.
- (C) Pour exécuter une action (ex : `setIsLoading(false)`) peu importe le succès ou l'échec.
- (D) Pour supprimer les données reçues afin de libérer de la mémoire.

- **Question 23 : Conversion des données**

Quelle méthode doit-on généralement appeler sur la réponse d'un **fetch** pour extraire les données au format objet JavaScript ?

- (A) `res.toObject()`
- (B) `res.json()`
- (C) `res.parse()`
- (D) `res.stringify()`

- **Question 21 : Gestion des erreurs HTTP avec **fetch****

Si un serveur répond avec une erreur ****404****, quel sera le comportement par défaut de **fetch** dans un bloc **try...catch** ?

- (A) Il lancera (**throw**) une exception capturée par le **catch**.
- (B) Il considérera l'appel comme réussi (pas d'exception) mais avec la propriété **ok** à **false**.
- (C) Il redémarrera automatiquement le navigateur.
- (D) Il retournera **undefined** sans rien dire.

- **Question 24 : Utilisation du **await****

Quelle est la condition obligatoire pour pouvoir utiliser le mot-clé **await** devant une promesse ?

- (A) La fonction parente doit être marquée comme **async**.
- (B) Le code doit être écrit à l'intérieur d'une boucle **for**.
- (C) On doit obligatoirement utiliser une variable **let**.
- (D) Le navigateur doit être en mode administrateur.

3 Étude de Cas : Débogage (30%)

L'application suivante tente de récupérer des produits et de les afficher. Elle contient **5 erreurs** de logique, de syntaxe ou de gestion d'asynchronisme.

Consigne : Identifiez les 5 erreurs et expliquez pourquoi elles empêchent le bon fonctionnement.

Fichier 1 : Produit.jsx

```
1 import React from 'react';
2
3 const Produit = ( nom, prix ) => {
4   return (
5     <div className="card">
6       <h2>{nom}</h2>
7       <p>Prix : {prix} $</p>
8     </div>
9   );
10 };
11 export default Produit;
```

Fichier 2 : App.jsx

```
1 import { useState, useEffect } from 'react';
2 import Produit from './Produit';
3
4 export default function App() {
5   const [items, setItems] = [];
6   const [error, setError] = useState(false);
7
8   useEffect(() => {
9     const fetchData = async () => {
10       try {
11         const res = fetch("https://api.exemple.com/items");
12         const data = await res.json();
13
14         data.push({ id: 999, title: "Promotion", price: 0 });
15         setItems(data);
16       } catch (err) {
17         setError(true);
18       }
19     };
20     fetchData();
21   });
22
23   return (
24     <main>
25       <h1>Ma Boutique</h1>
26
27       {error && <p>Une erreur est survenue</p> : <p>Chargement réussi</p>}
28
29       {items.map(item => (
30         <Produit key={item.id} nom={item.title} prix={item.price} />
31       ))}
32     </main>
33   );
34 }
```

Encerclez ou identifiez les 6 erreurs dans le code directement